

Chapter 9

Pagination & Sorting



Serving large result sets one page at a time with Spring Data JPA's `Page` and `Pageable` .

1-2. Why Pagination, and What is It?

If the database contains thousands or millions of records, returning all of them at once causes slow server performance, high database load, high network bandwidth usage, more memory consumption, and a slow response to the client.

Pagination solves this by returning data page by page — dividing a large collection of records into smaller pages:

```
Database contains 100 records.  
  
Page 0 → Records 1 to 10  
Page 1 → Records 11 to 20  
Page 2 → Records 21 to 30
```

3. What is Page?

`Page<T>` is an interface provided by Spring Data JPA. It represents one page of data along with useful pagination information, for example `Page<FoodDonation>`. A Page object contains:

content	totalElements	totalPages	size	number	first	last	numberOfElements	empty
---------	---------------	------------	------	--------	-------	------	------------------	-------

4. Meaning of Page Response Fields

FIELD	MEANING
content	Contains the actual records
number	Current page number (0 = first page)
size	Maximum number of records requested per page
numberOfElements	Actual number of records returned on the current page (e.g. requested 10, only 4 exist → 4)
totalElements	Total number of matching records in the database (e.g. 25 donations → 25)
totalPages	Total pages required — $\text{ceil}(\text{totalElements} / \text{size})$. With 25 records at size 10: Page 0 → 10, Page 1 → 10, Page 2 → 5, so totalPages = 3
first	True if the current page is the first page
last	True if the current page is the last page
empty	True if the current page contains no records

5-9. Pageable and PageRequest

`Pageable` is an **interface** that stores pagination information: Page Number, Page Size, and Sorting Information. Because it only defines a contract, interfaces cannot have objects — the following is invalid:

```
Pageable pageable = new Pageable(); // ✗ not allowed
```

Instead, the concrete class `PageRequest` implements `Pageable` :



We create a `Pageable` object manually using `PageRequest.of()` :

```
Pageable pageable = PageRequest.of(0,5);
```

This means Page Number = 0, Page Size = 5.

```
Pageable pageable = PageRequest.of(2,10);
```

This means the third page, 10 records per page — remember, pages start from 0.

10-11. Spring-Created vs Developer-Created Pageable

A Spring-Created Pageable

```
@GetMapping("/available")
public ResponseEntity<Page<FoodDonation>> getAvailableFood(Pageable pageable){

    return ResponseEntity.ok(
        foodDonationService.getAvailableDonations(pageable)
    );
}
```

Spring automatically creates the `Pageable` object from the query string — `?page=0&size=5&sort=id,asc` . The developer does not create it.

B Developer-Created Pageable

```
Pageable pageable = PageRequest.of(
    0,
    5,
    Sort.by("id").ascending()
);
```

Created manually, used for fixed pagination cases such as the latest 5 donations, top 10 products, or the latest 20 orders.

12-14. Repository, Service & Controller

REPOSITORY

```
// Old
List<FoodDonation> findByStatus(DonationStatus status);

// New
Page<FoodDonation> findByStatus(
    DonationStatus status,
    Pageable pageable
);
```

SERVICE

```
public Page<FoodDonation> getAvailableDonations(Pageable pageable){

    return foodDonationRepository.findByStatus(
        DonationStatus.AVAILABLE,
        pageable
    );
}
```

CONTROLLER

```
@GetMapping("/available")
public ResponseEntity<Page<FoodDonation>> getAvailableFood(Pageable pageable){

    return ResponseEntity.ok(
        foodDonationService.getAvailableDonations(pageable)
    );
}
```

SWAGGER REQUEST

```
GET /donations/available?page=0&size=5
```

16-19. Sorting

Sorting means arranging records in a specific order — Ascending goes small to large / A to Z; Descending goes large to small / Z to A.

REQUEST	MEANING
sort=id,asc	Sort by ID ascending
sort=id,desc	Sort by ID descending
sort=foodName,asc	Sort by food name ascending
sort=quantity,desc	Sort by quantity descending

Spring provides the `Sort` class:

```
Sort.by("id").ascending()
Sort.by("id").descending()
```

Combined with `PageRequest` :

```
Pageable pageable = PageRequest.of(
    0,
    5,
    Sort.by("id").ascending()
);
```

This means Page 0, 5 records, sorted by id ascending.

20. Flow of Pagination



21-23. Key Differences

LIST	PAGE
Contains only data	Contains data plus pagination metadata
No pagination information	Total pages, total elements, current page, page size, first/last info

PAGEABLE	PAGEREQUEST
Interface	Class
Defines the pagination contract	Implements Pageable
Cannot create an object	Object can be created, used by developers

SPRING-CREATED PAGEABLE	DEVELOPER-CREATED PAGEABLE
Created automatically by Spring	Created manually using PageRequest.of()
Uses page, size, and sort from the client request	Used when the developer wants fixed pagination

Interview Questions & Answers

1. Why do we need Pagination?

Returning thousands or millions of records at once causes slow server performance, high database load, high bandwidth usage, excessive memory consumption, and slow client responses. Pagination avoids this by returning data page by page.

2. What is Pagination?

Pagination is the process of dividing a large collection of records into smaller pages, e.g. Page 0 → records 1-10, Page 1 → records 11-20, and so on.

3. What is Page?

`Page<T>` is a Spring Data JPA interface representing one page of data along with pagination metadata such as content, totalElements, totalPages, size, number, first, last, numberOfElements, and empty.

4. What is Pageable?

Pageable is an interface that stores pagination information — page number, page size, and sorting information.

5. Why is Pageable an interface?

It only defines the contract for pagination information; the actual implementation is left to a concrete class, so different pagination strategies can implement it.

6. Why can't we create an object of Pageable?

Interfaces in Java cannot be instantiated directly (there is no constructor to call) — a concrete implementing class is required.

7. Which class implements Pageable?

`PageRequest` is the concrete class that implements the Pageable interface.

8. What is PageRequest?

PageRequest is a concrete class implementing Pageable, used when the developer wants to create a Pageable object manually, e.g. `PageRequest.of(0, 5)`.

9. Why do we use PageRequest.of()?

Because Pageable is an interface and cannot be instantiated directly, PageRequest.of() is the factory method used to create a Pageable object.

10. Difference between Page and List?

A List contains only the data itself with no pagination information. A Page contains the data plus metadata — total pages, total elements, current page, page size, and first/last page indicators.

11. Difference between Pageable and PageRequest?

Pageable is an interface defining the pagination contract and cannot be instantiated. PageRequest is the class that implements Pageable, and its objects can be created and used by developers.

12. Difference between Spring-created Pageable and Developer-created Pageable?

Spring-created Pageable is built automatically from the request's page, size, and sort query parameters. Developer-created Pageable is built manually with PageRequest.of() for fixed pagination needs, such as fetching the latest 5 donations.

13. How does Spring automatically create a Pageable object?

By declaring a Pageable parameter in a controller method, Spring reads the query parameters page, size, and sort from the incoming request and constructs the Pageable object for you.

14. What information does Pageable contain?

Page number, page size, and sorting information.

15. What does content represent in a Page response?

The actual list of records returned for the current page.

16. What is totalElements?

The total number of matching records present in the database across all pages, not just the current page.

17. What is totalPages?

The total number of pages required to display all matching records, calculated as `ceil(totalElements / size)`.

18. What is number?

The current page number (zero-indexed, so 0 means the first page).

19. What is size?

The maximum number of records requested per page.

20. What is numberOfElements?

The actual number of records returned on the current page, which may be less than size on the last page.

21. What is first?

A boolean that returns true if the current page is the first page.

22. What is last?

A boolean that returns true if the current page is the last page.

23. What is empty?

A boolean that returns true if the current page contains no records.

24. What is Sorting?

Sorting is arranging records in a specific order — ascending (small to large, A to Z) or descending (large to small, Z to A).

25. Difference between Ascending and Descending sorting?

Ascending arranges values from smallest to largest (or A to Z); descending arranges them from largest to smallest (or Z to A).

26. How do you sort records in Spring Data JPA?

Using the Sort class, e.g. `Sort.by("id").ascending()` or `Sort.by("id").descending()`, typically passed into a PageRequest.

27. How do you paginate data in Spring Data JPA?

By changing the repository method to accept and return a Pageable/Page, e.g. `Page<FoodDonation> findByStatus(DonationStatus status, Pageable pageable)`, and passing page/size query parameters from the client.

28. How do you paginate and sort data together?

By constructing a Pageable with both page/size and a Sort, e.g. `PageRequest.of(0, 5, Sort.by("id").ascending())`, or by including both page/size and sort query parameters when Spring builds the Pageable automatically.

29. What is the repository method signature for pagination?

`Page<FoodDonation> findByStatus(DonationStatus status, Pageable pageable);` — it accepts a Pageable parameter and returns a Page instead of a List.

30. Explain the complete flow of Pagination from client request to database.

The client sends a request with page, size, and optional sort parameters. The Controller receives a Pageable (built by Spring or manually). It passes this to the Service, which passes it to the Repository. The Repository queries the Database using the pagination and sorting information, and the result is wrapped and returned as a `Page<FoodDonation>` back up through Service and Controller to the Client.

CHAPTER SUMMARY

Pagination and sorting let FoodBridge serve large result sets efficiently by returning data one page at a time instead of loading every record into memory. Spring Data JPA's `Page<T>` carries both the data and rich metadata — total elements, total pages, and first/last indicators — while `Pageable`, backed by the concrete `PageRequest` class, describes exactly which page, size, and sort order the client wants.

Repository, service, and controller methods only need to accept a `Pageable` and return a `Page<T>` for Spring to handle the rest — whether the `Pageable` comes automatically from query parameters or is constructed manually for fixed pagination needs like "latest 5 donations."